

LAB#05

IMPLEMENTING UNINFORMED SEARCH TECHNIQUES

Implement shortest path using BFS and DFS search techniques.

I. Graph Traversals

Graph traversal means visiting every vertex and edge exactly once in a well-defined order. While using certain graph algorithms, you must ensure that each vertex of the graph is visited exactly once. The order in which the vertices are visited are important and may depend upon the algorithm or question that you are solving. During a traversal, it is important that you track which vertices have been visited. The most common way of tracking vertices is to mark them.

II. Breadth First Search (BFS)

In AI, breadth-first search (BFS) is a graph search algorithm that begins at the root node and explores all the neighboring nodes. Then for each of those nearest nodes, it explores their unexplored neighbor nodes, and so on, until it finds the goal. BFS is an uninformed search method that aims to expand and examine all nodes of a graph or combination of sequences by systematically searching through every solution. It does not use a heuristic algorithm.

From the standpoint of the algorithm, all child nodes obtained by expanding a node are added to a FIFO queue. In typical implementations, nodes that have not yet been examined for their neighbors are placed in some container (such as a queue or linked list) called “open” and then once examined are placed in the container “closed”.

Algorithm:

1. En-queue the root node.
2. De-queue a node and examine it.
3. If the element (required) is found in this node, quit the search and return a result.
4. Otherwise en-queue any successors (the direct child nodes) that have not yet been discovered.
5. If the queue is empty, every node on the graph has been examined – quit the search and return “not found”.
6. Repeat from Step 2.

Example:

```
graph = {
    'A' : ['B', 'C'],
    'B' : ['D', 'E'],
    'C' : ['F'],
    'D' : [],
    'E' : ['F'],
    'F' : []
}

visited = []
queue = []

def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)

    while queue:
        s = queue.pop(0)
        print (s, end = ' ')
        for neighbour in graph[s]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

bfs(visited, graph, 'A')
```

Output:

A B C D E F

Example:

```

import queue
graph = {'A':['B','C','E'], 'B':['E','D'],
        'C':['F','G'], 'D': ['B','E'], 'E':['B','D'], 'F': [], 'G': []}
openList = queue.Queue()
initial = 'A'
final = 'G'
openList.put(initial)
closeList = []
print("Open List\t\t\t\t\tClose List")
print(list(openList.queue),"\t\t\t\t\t",closeList)
while not openList.empty():
    s = openList.get()
    closeList.append(s)
    if s==final:
        print(list(openList.queue),"\t\t\t\t\t",closeList)
        break
    m = graph[s]
    for n in m:
        if closeList.count(n) <=0 and list(openList.queue).count(n) <=0:
            openList.put(n)
    print(list(openList.queue),"\t\t\t\t\t",closeList)

```

Output:

Open List	Close List
['A']	[]
['B', 'C', 'E']	['A']
['C', 'E', 'D']	['A', 'B']
['E', 'D', 'F', 'G']	['A', 'B', 'C']
['D', 'F', 'G']	['A', 'B', 'C', 'E']
['F', 'G']	['A', 'B', 'C', 'E', 'D']
['G']	['A', 'B', 'C', 'E', 'D', 'F']
[]	['A', 'B', 'C', 'E', 'D', 'F', 'G']

III. Depth First Search (DFS)

Depth-first search (DFS) is an algorithm for traversing or searching a tree, tree structure, or graph. One starts at the root (selecting some node as the root in the graph case) and explores as far as possible along each branch before backtracking. DFS is an uninformed search that progresses by expanding the first child node of the search tree that appears and thus going deeper and deeper until a goal node is found, or until it hits a node that has no children. Then the search backtracks, returning to the most recent node it hasn't finished exploring. In a non-recursive implementation, all freshly expanded nodes are added to a stack for exploration. It maintains the order in which the vertices are to be processed.

Algorithm:

1. In Depth first search put the starting vertex into the stack. Mark it as visited. Then display it.
2. If stack top node has adjacent unvisited vertex, then visit the adjacent unvisited vertex and mark it visited. Push it into the stack and display it.
3. Else pop the top element out of the stack.
4. Repeat step-2 until stack is empty.

Example:

```
graph = {
    'A' : ['B', 'C'],
    'B' : ['D', 'E'],
    'C' : ['F'],
    'D' : [],
    'E' : ['F'],
    'F' : []
}

visited = []

def dfs(visited, graph, node):
    if node not in visited:
        print (node)
        visited.append(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

dfs(visited, graph, 'A')
```

Output:

```
A
B
D
E
F
C
```

Example:

```

import queue

graph = {'A': ['B', 'C', 'E'], 'B': ['A', 'E', 'D'],
        'C': ['F', 'G'], 'D': ['B', 'E'], 'E': ['B', 'D'], 'F': [], 'G': []}

stack = queue.LifoQueue()
initial = 'A'
goal = 'G'
stack.put(initial)
closedList = []
openList = [goal]
print("Open List\t\tClose List")
while not stack.empty():
    node = stack.get()
    openList.pop(0)
    if node==goal:
        print(openList,end="\t\t")
        closedList.append(node)
        print(closedList)
        break
    items = graph[node]
    items.reverse()
    for item in items:
        if openList.count(item) <=0 and closedList.count(item) <=0:
            stack.put(item)
            openList.insert(0, item)
    closedList.append(node)
    print(openList, end="\t\t\t")
    print(closedList)

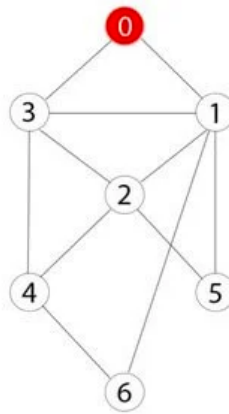
```

Output:

Open List	Close List
['B', 'C', 'E']	['A']
['D', 'C', 'E']	['A', 'B']
['C', 'E']	['A', 'B', 'D']
['F', 'G', 'E']	['A', 'B', 'D', 'C']
['G', 'E']	['A', 'B', 'D', 'C', 'F']
['E']	['A', 'B', 'D', 'C', 'F', 'G']

Activity- 1:

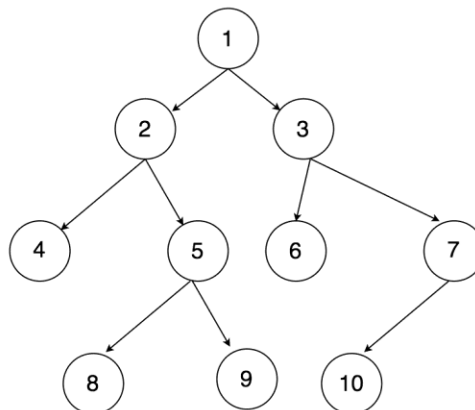
Consider the following graph. If there is ever a decision between multiple neighbor nodes in the BFS algorithm, assume we always choose the letter closest to the beginning of the alphabet first. A connected graph with 7 nodes and 7 edges. The edges are undirected and unweight. Distance between two nodes will be measured based on the number of edges separating two vertices. Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors.



Define function name `_connected_component`, this function keep track of all the visited nodes with BFS, is as simple as implementing the steps of the algorithm and assign `_queue` variable already has a node to be checked, i.e., the starting vertex that is used as an entry point to explore the graph. The next step is to implement a loop that keeps cycling until queue is empty. At each iteration of the loop, a node is checked. If this wasn't visited already, its neighbors are added to queue. Once the loop is exited, the function (`connected_component`) returns all of the visited nodes.

Activity- 2:

Apply Breadth First Search on following graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.



Activity- 3:

Repeat Activity-2, apply BFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

Activity- 4:

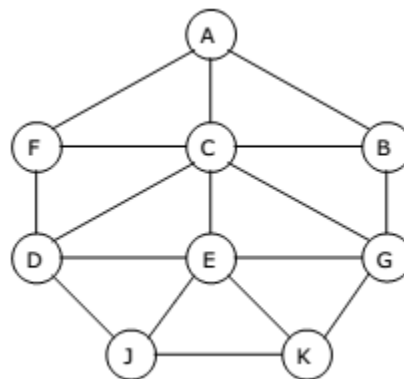
Apply Depth First Search on Activity-2 graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.

Activity- 5:

Repeat Activity-4, apply DFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

Activity- 6:

Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Breadth First Search on following graph considering the initial state is A and final state is K. Also evaluate it manually.

**Activity- 7:**

Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Depth First Search on Activity-6 graph considering the initial state is A and final state is K. Also evaluate it manually.